

Shadowseed: Remembering Without Trusting

A Validation-Gated Memory Architecture for Language Model Systems

H. Visser

E-AI-MODEL

August 2026

Abstract

Long-lived language-model systems need memory, but persistence is not the same as warrant. A generated suspicion, missing dependency, alternative hypothesis, unresolved contradiction, or relevant what-if can be worth remembering while still being unfit to steer an answer. Shadow Seed Learning (SSL) treats such items as bounded epistemic candidates rather than facts. New seeds are remembered with positive trace and zero steering weight. Authority is governed separately through typed Validation Gate decisions, contradiction state, source-aware evidence accounting, lifecycle rules, and a current point-of-use authorization. External support is deduplicated by the underlying source reference across evidence channels, so relabeling one source cannot manufacture independent authority. Semantic recurrence is observation-scoped: cluster membership alone is not recurrence, and one detector observation contributes at most one recurrence credit to a semantic candidate. Detector output produced after exposure to SSL-derived context remains auditable but cannot count as independent same-turn recurrence. The product surface is an ordinary evidence-backed chat. A tester may optionally request an automatically generated same-message SSL-off control that uses the same model configuration and visible pre-turn history but does not receive surfaced seeds, enter detection, alter recurrence, invoke the Gate, or mutate conversation state. This paper specifies the architecture and its executable contracts. It does not claim that SSL improves answer quality, detects missing information reliably, establishes semantic truth, provides universal security, or is production-ready. The reviewed reference implementation is source version 0.5.0 at commit 745ea31486f3d287a1f2bd97a7896f92a8ed6257 [1].

1 Introduction

Memory mechanisms are becoming a standard component of language-model agents. They can preserve conversational facts, retrieve documents, summarize past experience, or learn policies for storage and recall [2–7]. These capabilities solve a persistence problem. They do not, by themselves, solve an authority problem: what should happen when the remembered object was uncertain when it was created?

That case is common. A model may suspect that an answer omitted a causal qualification. It may notice that a boundary condition is missing, that two claims depend on an unstated assumption, that a contradiction deserves investigation, or that an alternative hypothesis should remain available. Clarification work shows that underspecified inputs materially degrade question answering and that asking for missing information remains difficult [8–11]. AbsenceBench and QuestBench similarly motivate explicit treatment of missing or unavailable information [12, 13]. None of these observations justifies turning a model-generated candidate into a fact merely because it was stored.

SSL starts from a stricter separation:

A system may remember a candidate without granting it authority to steer a later answer or action.

This gives uncertain memory an explicit epistemic status. A seed is a bounded candidate for later validation, not a truth claim. Its persistence is represented by trace. Its steering permission is represented separately by weight and derived authorization state. Newly created seeds are weightless. Validation signals are typed. Contradictions are first-class records. External evidence preserves source identity. Authorization is checked again at the point of use.

The distinction matters because memory can amplify errors. Retrieval systems can be poisoned [14]; tool-using agents can be redirected by untrusted content [15]; and recent work demonstrates attacks that persist through long-term agent memory [16]. A design in which retrieval, repetition, or storage automatically creates authority therefore combines a useful capability with an avoidable trust escalation.

This paper makes four contributions. First, it defines the epistemic object stored by SSL and separates candidate identity, persistence, and steering authority. Second, it specifies authority-changing and authority-blocking contracts, including source-level evidence identity, observation-scoped recurrence, contradiction handling, lifecycle, and point-of-use checks. Third, it defines a chat-first product contract with an optional same-message SSL-off control that cannot contaminate the live state. Fourth, it states a narrow evidence boundary: the current artifact supports implementation-contract claims only. General answer-quality improvement and semantic correctness remain research questions.

2 Problem setting and design goals

2.1 The stored object is a bounded epistemic candidate

The motivating case is a candidate omission, such as “the answer does not establish whether the association is causal.” SSL is deliberately broader than omissions. A seed can represent one bounded epistemic candidate of a type such as:

- a suspected gap or missing dependency;
- a bounded doubt or unresolved boundary condition;
- a missing relation between otherwise present claims;
- an assumption that may need validation;
- an alternative hypothesis;
- a contradiction that should be investigated; or
- a relevant what-if direction that should remain available for testing.

The candidate type is audit metadata. It does not create authority. A missing-dependency seed and an alternative-hypothesis seed begin with the same epistemic constraint: neither may steer merely because the detector generated it.

A seed should be atomic enough to validate or reject as one bounded candidate. Atomicity is a tested normalization heuristic, not a semantic guarantee. Model-generated candidates can still be vague, compound, redundant, or wrong. The architecture therefore avoids deriving steering permission from fluency, detector confidence, semantic similarity, or candidate type alone.

2.2 Failure modes

Persistence mistaken for confidence. A detector can repeat the same error. Reappearance is an observation about recurrence, not proof that the candidate is correct.

Similarity mistaken for recurrence. Several outputs from one detector pass may cluster together. Treating every member as an independent recurrence would let one observation create artificial reinforcement.

Retrieval mistaken for validation. A retrieved item can be irrelevant, duplicated, poisoned, or derived from the same underlying source through another channel. Retrieval is an input to validation, not a verdict.

Same-source relabeling mistaken for independence. If one source is observed as both “retrieval” and “trusted source,” counting two authority credits rewards transport metadata rather than independent support.

Historical authorization mistaken for current permission. A seed can later be contradicted, expire, or become irrelevant. A cached prior authorization cannot substitute for a current point-of-use decision.

Model self-exposure mistaken for independent evidence. If a model sees SSL-derived content and then generates a matching candidate, that output is contaminated by the system’s own intervention. It may be logged, but it cannot count as independent same-turn recurrence.

3 Related work

3.1 Retrieval and long-term memory

RAG and REALM established influential patterns for bringing external information into model context [2, 3]. Self-RAG and Adaptive-RAG add learned or conditional retrieval behavior [17, 18]. MemGPT organizes limited context through explicit memory tiers [4], while Reflexion stores linguistic feedback from prior attempts [19].

Recent work broadens memory management itself. MemBench evaluates memory effectiveness, efficiency, and capacity across factual and reflective memory settings [5]. AgeMem learns when to store, retrieve, update, summarize, and discard information as agent actions [6]. EMA adds selective episodic-memory filtering to reduce redundant context [7]. These approaches ask what to remember and how to use remembered information. SSL isolates a complementary question: under what conditions may an explicitly uncertain remembered candidate acquire bounded steering authority?

3.2 Missing information, underspecification, and clarification

AbsenceBench studies information that has been deliberately removed and reports a gap between finding present information and identifying what is missing [12]. QuestBench evaluates acquisition of missing variables in reasoning tasks [13]. More recent work finds that underspecified questions are common in QA datasets and that resolving underspecification improves performance [8]. AskBench evaluates when and what a model should ask under missing or misleading information [9]. Structured-uncertainty approaches provide explicit criteria for selecting clarification questions in tool-using agents [10], and factual clarification work shows continued difficulty eliciting missing information in multi-hop settings [11].

SSL does not replace clarification. A seed may lead to a question, a retrieval request, or no action at all. The architecture governs whether the candidate is authorized to affect that choice.

3.3 Verification, uncertainty, and memory security

Chain-of-Verification separates draft generation from targeted verification [20]. Semantic entropy estimates uncertainty over generated meanings [21]. Both are relevant signals, but neither is identical to system-level

authority. A low-uncertainty statement can be unsupported, and a high-uncertainty candidate can still be worth investigating.

Security work gives a second reason to separate retrieval from authority. PoisonedRAG demonstrates knowledge-corruption attacks on retrieval-augmented systems [14]; AgentDojo evaluates indirect prompt injection against tool-using agents [15]; and Visual Inception demonstrates a long-term-memory poisoning threat in agentic recommenders [16]. SSL does not claim to solve these attacks. It aims to keep a specific trust boundary explicit so that persistence and retrieval do not silently become permission to steer.

4 Shadow Seed Learning

4.1 Seed state

Let a seed at time t be represented by

$$s_t = (x, c, \tau_t, w_t, z_t, E_t, C_t, v_t), \quad (1)$$

where x is candidate content, c is candidate-type metadata, $\tau_t \geq 0$ is trace, $w_t \in [0, 1]$ is steering weight, z_t is lifecycle/authority status, E_t records accepted external evidence identities, C_t records contradictions, and $v_t \in \mathbb{N}_0$ is the authority version.

Creation obeys

$$\tau_{t_0} > 0, \quad w_{t_0} = 0. \quad (2)$$

Therefore

$$\tau_t > 0 \not\Rightarrow w_t > 0. \quad (3)$$

A seed can remain remembered indefinitely while still having no steering authority.

The reference lifecycle uses NEW, ACTIVE, DECAYING, DORMANT, PROMOTED, and EXPIRED. Expiry is terminal in the supported lifecycle. Deterministic expiry may mechanically clear authority as part of lifecycle enforcement; this is not a new positive policy decision. Likewise, restoration from a previously validated durable state restores recorded authority state rather than inventing a new Gate verdict. New positive authority decisions remain Gate-controlled.

4.2 Trace and recurrence

Trace represents remembered presence and reactivation, not truth. Decay can reduce trace without changing the candidate’s content. Dormant candidates can be reactivated by trigger or semantic matching, subject to lifecycle rules.

Semantic recurrence is scoped to detector observations. Let o denote one detector observation context and let $M(o)$ be the set of normalized candidates produced in that observation. If multiple members of $M(o)$ map to the same semantic seed or cluster, the observation contributes at most one recurrence credit for that seed:

$$r(s, o) \in \{0, 1\}. \quad (4)$$

For a sequence of independent observations O , recurrence is

$$R(s, O) = \sum_{o \in O} r(s, o). \quad (5)$$

Cluster membership alone is not recurrence. The count refers to observations, not the number of textual variants emitted inside one observation.

The independence qualification matters. A detector observation produced after the model has been exposed to SSL-surfaced content is marked contaminated for same-turn recurrence accounting. It remains auditable and may be reconsidered in a later independent observation, but it cannot earn independent same-turn recurrence credit. This prevents a surfaced seed from creating evidence for itself through the model’s subsequent output.

4.3 External evidence identity

Validation signals are typed for provenance and policy handling. Examples include trusted-source evidence, retrieval results, human feedback, probes, task outcomes, recurrence, contradiction, and contradiction resolution. Signal kind records how information entered the system; it is not the underlying authority identity.

For external evidence signal e , define the authority identity as

$$\kappa(e) = \text{source_ref}(e). \quad (6)$$

A non-empty stable source reference is required for external support to count toward authority. Replaying the same source reference is idempotent. Relabeling one underlying source under another external signal kind does not create another authority credit. Thus

$$\kappa(e_1) = \kappa(e_2) \Rightarrow \text{at most one accepted authority credit}, \quad (7)$$

for positive external support derived from the same underlying source.

This identity rule does not prove source independence or truth. Two distinct URLs can still reflect one upstream report, and one source can still be wrong. Source-level deduplication is an accounting boundary, not a semantic-verification theorem.

4.4 Validation Gate and contradiction

A named Gate policy consumes a read-only authority snapshot A_t and a set of typed signals Σ_t :

$$G(P, A_t, \Sigma_t) \rightarrow (A_{t+1}, g_t), \quad (8)$$

where P names the policy and g_t is an immutable Gate event. The event records the offered signals, accepted and rejected inputs, before-and-after authority state, reason, contradiction state, and authority version.

The Gate owns policy decisions that grant, reduce, block, or otherwise change steering authority in response to validation inputs. This claim is intentionally scoped. Mechanical lifecycle expiry can clear authority without a new policy decision, and durable-state restoration can reinstate a previously recorded validated state. Unsupported test hooks are outside the product contract and cannot be used to redefine runtime authority semantics.

Contradictions are explicit blocking records. An unresolved contradiction prevents positive influence even when prior support exists. Resolution is itself typed and auditable. The architecture therefore avoids reducing support and contradiction to a single scalar confidence number that could hide why authority is blocked.

4.5 Authority version and point-of-use authorization

Authority-relevant transitions advance a version v_t . A prior authorization is valid only for the authority snapshot it evaluated. When a seed is about to affect retrieval, prompt construction, an answer, a warning, a probe, or another downstream action, the runtime checks current state again.

The point-of-use predicate can be written abstractly as

$$U(s_t, q_t, a_t) \rightarrow \{\text{allow}, \text{deny}\}, \quad (9)$$

where q_t is the current request context and a_t is the proposed influence action. A positive result requires current policy eligibility, no unresolved blocking contradiction, a non-expired lifecycle state, and relevance to the proposed action. Both allowed and denied attempts are logged.

This second check is essential. Promotion is not permanent truth. Retrieval is not confirmation. Surfacing is not an instruction. A seed can be remembered and even previously promoted while still being denied for the current action.

5 Product contract

5.1 Chat-first live path

The normal product surface is a persistent ordinary chat. The live path is evidence-backed: recurrence alone does not grant live steering authority. Answers follow the language of the current user question rather than a repository-wide language default.

A live turn may surface only seeds that pass the current point-of-use contract. If no seed is authorized, SSL may still detect and remember candidates after the answer, but those candidates cannot retroactively influence the answer that created them.

5.2 Optional same-message SSL-off control

A tester may request a paired comparison for the same message. The system automatically generates a control response with SSL disabled. The tester does not author a baseline. The control receives:

- the same model configuration;
- the same user message; and
- the same visible conversation history from before the turn.

The control does not receive surfaced seeds. It is not passed through candidate detection. It creates no recurrence credit, invokes no Validation Gate decisions, and is not appended to the live conversation history. Only the real live turn mutates state.

This design reduces one class of comparison contamination. It does not make the pair a blinded randomized experiment. Stochastic generation, hosted-model changes, latency, and provider-side nondeterminism can still differ between calls. A visible answer difference may be attributed to SSL only when at least one authorized seed was actually surfaced into the live generation path. Even then, the pair demonstrates a local behavioral difference, not a general performance benefit.

6 Auditability and artifact boundaries

The reference implementation separates four kinds of artifact authority. The software source of truth defines executable behavior. The model/architecture specification explains intended contracts. Decision logs capture runtime evidence. Validation artifacts and benchmarks support bounded claims. Execution plans and historical migration reports do not override current runtime or accepted architecture decisions [1].

The reviewed implementation for this manuscript is source version 0.5.0 at commit 745ea31486f3d287a1f2bd97a7896f92a8ed6257. The version string identifies the source tree. It is not, by itself, evidence that

a public binary release with the matching tag or assets exists. The manuscript source revision is a separate publication artifact and should be cited by its own repository commit when archived.

Audit records are designed to answer questions such as: What candidate was created? Which observation produced recurrence? Which underlying sources counted as evidence? Was a contradiction open? Which policy made the authority decision? Which authority version was checked at point of use? Was influence allowed or denied? This is an accountability objective, not a guarantee that the recorded evidence was semantically correct.

7 Executable assurance and claim boundary

The current artifact is evaluated primarily through executable contracts rather than open-ended answer-quality experiments. The repository includes unit and integration tests, multi-version Python CI, linting, build checks, branch-coverage enforcement, Workbench tests, portability checks, and standalone build checks [1]. These gates are useful for detecting contract regressions such as accidental authority mutation, evidence double-counting, unsupported recurrence, stale authorization, conversation-control contamination, or packaging drift.

This assurance supports claims of the form “the tested implementation enforces this contract under covered conditions.” It does not support stronger claims such as “the detector finds all important missing information” or “SSL improves answer quality.” In particular, the following remain outside the evidence provided by the current artifact:

- semantic correctness and completeness of generated candidates;
- calibrated utility of recurrence thresholds across domains;
- truth or independence of external sources;
- general answer-quality improvement over an SSL-off system;
- robustness to all prompt-injection, retrieval-poisoning, or memory-poisoning attacks;
- operational production readiness at scale; and
- human-factor effects of warnings, clarifications, or surfaced uncertainty.

A future answer-quality evaluation should freeze an implementation commit, define tasks before inspection of outcomes, use paired or randomized controls with reproducible model settings where possible, separate candidate quality from authority-policy effects, blind human review when feasible, report failure cases as well as gains, and preserve the distinction between statistical recurrence and external evidence.

8 Discussion

SSL can be viewed as an authority layer over uncertain memory. This framing is intentionally narrower than a general memory architecture. It does not decide every storage policy and does not require one detector or one retrieval backend. Its central invariant is that each module preserves the candidate’s epistemic identity. A gap must not silently become a conclusion; a what-if must not become a fact; recurrence must not become evidence; similarity must not become truth; retrieval must not become confirmation; promotion must not become permanent truth; and surfacing must not become an instruction.

This invariant also constrains stronger models. A stronger detector may produce better candidates, and a stronger verifier may produce better evidence assessments. Neither receives extra authority merely because of model capability. Authority remains a policy-governed system property.

The same principle provides a useful way to position SSL relative to recent learned-memory systems. AgeMem and EMA optimize memory actions and filtering [6, 7]. SSL does not compete with those mechanisms on storage efficiency. It can instead sit beside or above them as a trust contract for uncertain candidates. Likewise, clarification systems can decide what question is most valuable [9, 10]; SSL can govern whether a remembered hypothesis is authorized to trigger that question in the current context.

9 Limitations

The architecture introduces additional state, policy logic, logging, and tests. These create engineering cost. The source-reference identity rule is intentionally conservative but cannot detect hidden dependence among distinct sources. Observation-scoped recurrence prevents obvious within-observation inflation but does not prove that separate observations are causally independent. Candidate normalization can merge distinct ideas or split one idea into several seeds. Point-of-use relevance is itself a model or rule-based judgment and can be wrong. Hosted model APIs can change behavior over time. The optional SSL-off control doubles model calls for compared turns and therefore changes cost and latency.

The present implementation is research-ready in the bounded sense used in the repository: contracts are explicit and testable. It should not be described as production-ready without additional security review, scale testing, incident handling, operational monitoring, data-governance analysis, and domain-specific evaluation.

10 Conclusion

Shadow Seed Learning separates remembering from trusting. Its basic object is a bounded epistemic candidate that can persist with zero steering weight. Authority is added only through explicit policy and remains defeasible through contradiction, lifecycle, and point-of-use checks. External evidence is deduplicated by underlying source identity across channels. Semantic recurrence is counted across detector observations rather than textual cluster members, and contaminated same-turn observations cannot reinforce themselves. The chat-first product contract keeps live state separate from an automatically generated SSL-off comparison. These design choices make uncertain memory inspectable without pretending that persistence is proof. The current contribution is an architecture and executable contract, not evidence of general answer-quality improvement.

LLM usage disclosure

Large language models, including ChatGPT and other model systems used during the Shadowseed project, assisted with ideation, drafting, editing, literature discovery, code review, and implementation analysis. They are not listed as authors. The human author is responsible for the manuscript, source verification, claims, citations, and released artifact.

Artifact note

The implementation described here is available in the Shadowseed Pro repository [1]. The implementation anchor for this manuscript is commit 745ea31486f3d287a1f2bd97a7896f92a8ed6257. The paper source and compiled PDF are publication artifacts and should be rebuilt together whenever the manuscript changes.

References

- [1] E-AI-MODEL. *Shadowseed Pro*. Software research artifact. Source version 0.5.0; implementation reviewed at commit 745ea31486f3d287a1f2bd97a7896f92a8ed6257. The source version does not by itself imply availability of a public binary release. 2026. URL: <https://github.com/E-AI-MODEL/shadowseed-pro>.
- [2] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020. URL: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [3] Kelvin Guu et al. “Retrieval Augmented Language Model Pre-Training”. In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 3929–3938. URL: <https://proceedings.mlr.press/v119/guu20a.html>.
- [4] Charles Packer et al. “MemGPT: Towards LLMs as Operating Systems”. In: (2023). DOI: [10.48550/arXiv.2310.08560](https://arxiv.org/abs/2310.08560).
- [5] Haoran Tan et al. “MemBench: Towards More Comprehensive Evaluation on the Memory of LLM-based Agents”. In: *Findings of the Association for Computational Linguistics: ACL 2025*. Vienna, Austria: Association for Computational Linguistics, July 2025, pp. 19336–19352. DOI: [10.18653/v1/2025.findings-acl.989](https://arxiv.org/abs/2025.findings-acl.989). URL: <https://aclanthology.org/2025.findings-acl.989/>.
- [6] Yi Yu et al. “Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents”. In: *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. San Diego, California, United States: Association for Computational Linguistics, July 2026, pp. 21457–21483. DOI: [10.18653/v1/2026.acl-long.981](https://arxiv.org/abs/2026.acl-long.981). URL: <https://aclanthology.org/2026.acl-long.981/>.
- [7] Hongyi Lan et al. “EMA: An Episodic Memory Agent for Efficient and Selective Memory”. In: *Findings of the Association for Computational Linguistics: ACL 2026*. San Diego, California, United States: Association for Computational Linguistics, July 2026, pp. 5088–5102. DOI: [10.18653/v1/2026.findings-acl.250](https://arxiv.org/abs/2026.findings-acl.250). URL: <https://aclanthology.org/2026.findings-acl.250/>.
- [8] Yunchong Huang, Gianni Barlacchi, and Sandro Pezzelle. “Who is the richest club in the championship? Detecting and Rewriting Underspecified Questions Improve QA Performance”. In: *Findings of the Association for Computational Linguistics: ACL 2026*. San Diego, California, United States: Association for Computational Linguistics, July 2026, pp. 26266–26281. DOI: [10.18653/v1/2026.findings-acl.1309](https://arxiv.org/abs/2026.findings-acl.1309). URL: <https://aclanthology.org/2026.findings-acl.1309/>.
- [9] Jiale Zhao, Ke Fang, and Lu Cheng. “When and What to Ask: AskBench and Rubric-Guided RLVR for LLM Clarification”. In: *Findings of the Association for Computational Linguistics: ACL 2026*. San Diego, California, United States: Association for Computational Linguistics, July 2026, pp. 17120–17140. DOI: [10.18653/v1/2026.findings-acl.845](https://arxiv.org/abs/2026.findings-acl.845). URL: <https://aclanthology.org/2026.findings-acl.845/>.
- [10] Manan Suri et al. “Structured Uncertainty guided Clarification for LLM Agents”. In: *Findings of the Association for Computational Linguistics: ACL 2026*. San Diego, California, United States: Association for Computational Linguistics, July 2026, pp. 40811–40838. DOI: [10.18653/v1/2026.findings-acl.2028](https://arxiv.org/abs/2026.findings-acl.2028). URL: <https://aclanthology.org/2026.findings-acl.2028/>.
- [11] Matthew Toles, Yukun Huang, and Zhou Yu. “Learning and Evaluating Factual Clarification Question Generation Without Examples”. In: *Proceedings of the Fourth Workshop on Generation, Evaluation and Metrics (GEM2)*. Vienna, Austria and virtual meeting: Association for Computational Linguistics, July 2025, pp. 200–211. URL: <https://aclanthology.org/2025.gem-1.15/>.

- [12] Harvey Yiyun Fu et al. “Absence Bench: Language Models Can’t See What’s Missing”. In: *Advances in Neural Information Processing Systems*. Vol. 38. Datasets and Benchmarks Track. 2025. URL: https://proceedings.neurips.cc/paper_files/paper/2025/hash/36b31e1bb8ecd4f4081686448e9eff2d-Abstract-Datasets_and_Benchmarks_Track.html.
- [13] Belinda Z. Li, Been Kim, and Zi Wang. “QuestBench: Can LLMs Ask the Right Question to Acquire Information in Reasoning Tasks?” In: *arXiv preprint arXiv:2503.22674* (2025). DOI: [10.48550/arXiv.2503.22674](https://doi.org/10.48550/arXiv.2503.22674).
- [14] Wei Zou et al. “PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models”. In: *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, Aug. 2025, pp. 3827–3844. URL: <https://www.usenix.org/conference/usenixsecurity25/presentation/zou-poisonedrag>.
- [15] Edoardo DeBenedetti et al. “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents”. In: *Advances in Neural Information Processing Systems*. Vol. 37. Datasets and Benchmarks Track. 2024. DOI: [10.52202/079017-2636](https://doi.org/10.52202/079017-2636).
- [16] Jiachen Qian. “Visual Inception: Compromising Long-term Planning in Agentic Recommenders via Multimodal Memory Poisoning”. In: *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. San Diego, California, United States: Association for Computational Linguistics, July 2026, pp. 20846–20862. DOI: [10.18653/v1/2026.acl-long.954](https://doi.org/10.18653/v1/2026.acl-long.954). URL: <https://aclanthology.org/2026.acl-long.954/>.
- [17] Akari Asai et al. “Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection”. In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=hSyW5go0v8>.
- [18] Soyeong Jeong et al. “Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity”. In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Mexico City, Mexico: Association for Computational Linguistics, 2024, pp. 7036–7050. DOI: [10.18653/v1/2024.naacl-long.389](https://doi.org/10.18653/v1/2024.naacl-long.389).
- [19] Noah Shinn et al. “Reflexion: Language Agents with Verbal Reinforcement Learning”. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023. DOI: [10.52202/075280-0377](https://doi.org/10.52202/075280-0377).
- [20] Shehzaad Dhuliawala et al. “Chain-of-Verification Reduces Hallucination in Large Language Models”. In: *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 3563–3578. DOI: [10.18653/v1/2024.findings-acl.212](https://doi.org/10.18653/v1/2024.findings-acl.212).
- [21] Sebastian Farquhar et al. “Detecting Hallucinations in Large Language Models Using Semantic Entropy”. In: *Nature* 630 (2024), pp. 625–630. DOI: [10.1038/s41586-024-07421-0](https://doi.org/10.1038/s41586-024-07421-0).